Screenshots — Terraform Variables

Lecture 6 · 40 shots, chronological · source stills from the session (2026-09-28 22:15 – 2026-09-29 00:47)

Terraform variables

Variables in Terraform are a great way to define centrally controlled reusable values. The information in Terraform variables is saved independently from the deployment plans, which makes the values easy to read and edit from a single file.

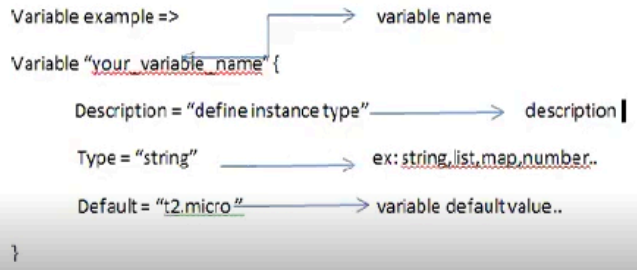
We have 3 types of variables:

1. **Input variables**: Sends values to Terraform.
2. **Output variables**: Retrieves values from Terraform.
3. **Local variables**: A local value assigns a name to an expression (*Expressions* refer to or compute values within a configuration Like hello or "5"), so you can use the name multiple times within a module instead of repeating the expression.

Input variables

Input variables are usually defined by stating a name, type and default value. However, the type and default values are not strictly necessary.

We use input variables to pass certain values from outside of the configuration or module.Variables can be predetermined in a file or included in the command-line options.



The variable declaration can optionally include three arguments:

Terraform variables serve as parameters for your Infrastructure as Code (IaC) configurations, acting like function arguments to make your code dynamic, flexible, and reusable. Instead of hardcoding properties like environment names, IP ranges, or machine sizes, you use variables to pass custom values at runtime. HashiCorp Developer +3

Three Core Types of Values

Terraform relies on three distinct concepts to handle dynamic data: YouTube · Tech Tutorials with Piyush +2

- **Input Variables**: Injected from outside the module to customize behavior (the primary focus of "variables").
- **Local Values (`locals`)**: Internal temporary variables used for computing private formulas and expressions within the code.
- **Output Values**: Returned values exposing internal resource details to the CLI or other dependent modules. ControlMonkey +4

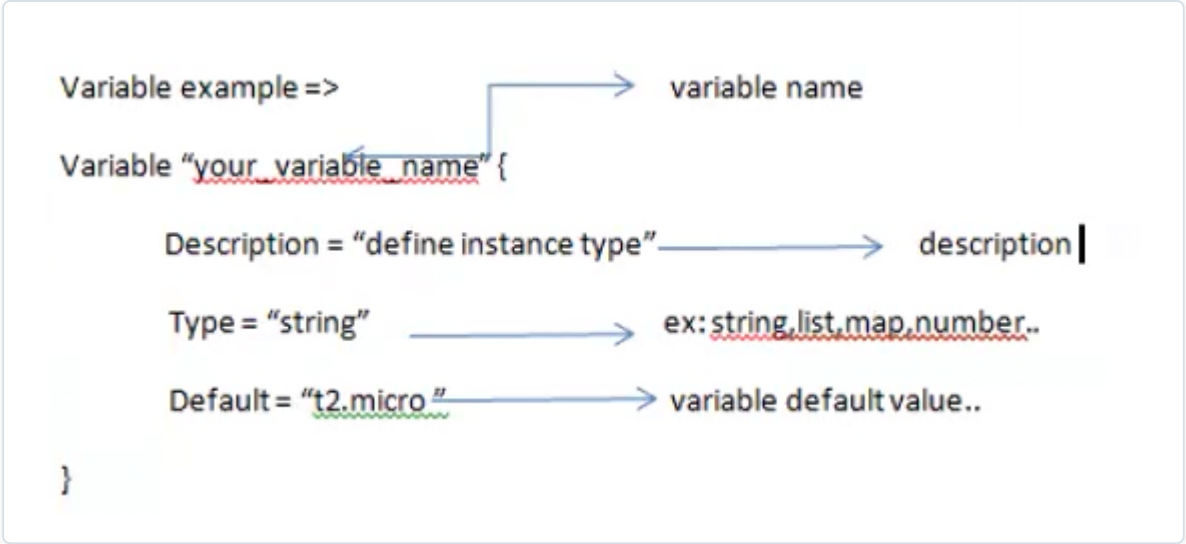
Declaring Input Variables

Variables are typically organized inside a dedicated `variables.tf` file. They use the `variable` block and support several configuration attributes: HashiCorp Developer +2

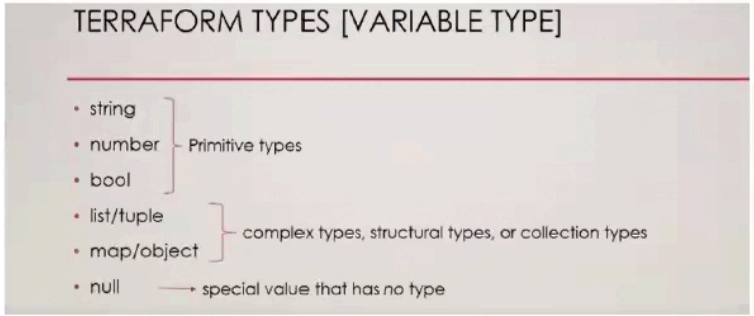
hcl

```hcl
variable "instance_type" {
  type        = string
  description = "The size of the virtual machine"
  default     = "t2.micro"
}

variable "database_password" {
  type        = string
  description = "The master password for the DB cluster"
  sensitive   = true  # Redacts values from CLI outputs and logs
}
```


Variable example =>
variable name
Variable "your_variable_name" {
Description = "define instance type"
description
Type = "string"
ex: string,list,map,number..
Default = "t2.micro"
variable default value..
}

Input variables **support multiple data types**. They are broadly categorized as simple and complex. `string`, `number`, `bool` are simple data types, whereas `list` and `map` are complex data types.



Here are some examples of how each type are defined and used.

Primitive Types

A primitive type is a simple type that isn't made from any other type. The available primitive types are:

1. **string**: a sequence of characters representing some text, such as "hello".
2. **number**: a numeric value. The number type can represent whole numbers like 10.
3. **bool**: either true or false..

Complex Types

A *complex* type is a type that groups multiple values into a single value. These values could be of a similar type or different types.

List:

Another type of Terraform variables lists. They work much like a numbered catalogue of values. Each value can be called by its corresponding index in the list. Here is an example of a list variable definition.

```
variable "users" {
  type    = list
  default = ["root", "user1", "user2"]
}
```

Lists can be used in the resource plans similarly to strings, but you'll also need to denote the index of the value you are looking for.

```
username = var.users[0]
```

Map:

Maps are a collection of string keys and string values. These can be useful for selecting values based on predefined parameters such as the server configuration by the monthly price.

```
variable "plans" {
  type = map
  default = {
    "5USD"  =  "1xCPU-1GB"
    "10USD" = "1xCPU-2GB"
    "20USD" = "2xCPU-4GB"
  }
}
```

You can access the right value by using the matching key. For example, the variable below would set the plan to "1xCPU-1GB".

Assign Values To Input Variables

There are multiple ways to assign values to variables. The following is the descending order of precedence in which variables are considered.

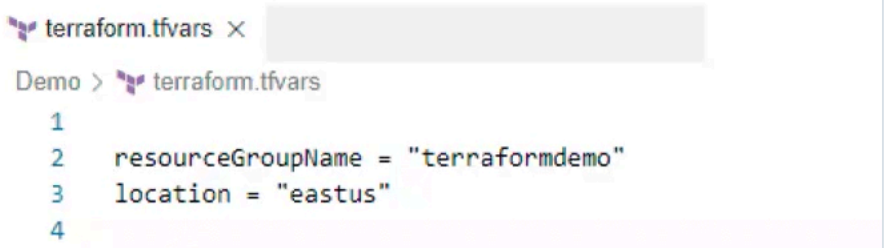
1. Command-line flags

The most simple way to assign value to a variable is using the -var option in the command line when running the terraform plan and terraform apply commands.

```
$ terraform apply  -var="location=eastus"
```

2. Variable Definition (.tfvars) Files

If there are many variable values to input, we can define them in a variable definition file. Terraform also automatically loads a number of variable definitions files if they are present:



3. Environment Variables

Terraform searches the environment of its own process for environment variables named **TF_VAR_<var-name>** followed by the name of a declared variable. Terraform scans all variables starting with TF_VAR and use those as variable values for Terraform.

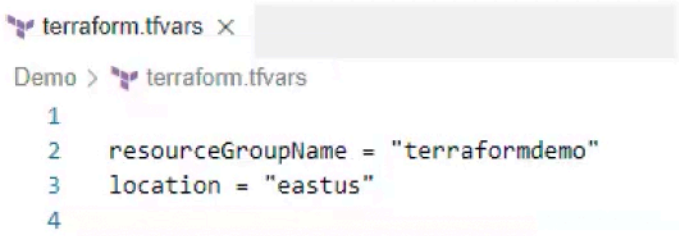
1. Command-line flags

The most simple way to assign value to a variable is using the -var option in the command line when running the terraform plan and terraform apply commands.

```
$ terraform apply  -var="location=eastus"
```

2. Variable Definition (.tfvars) Files

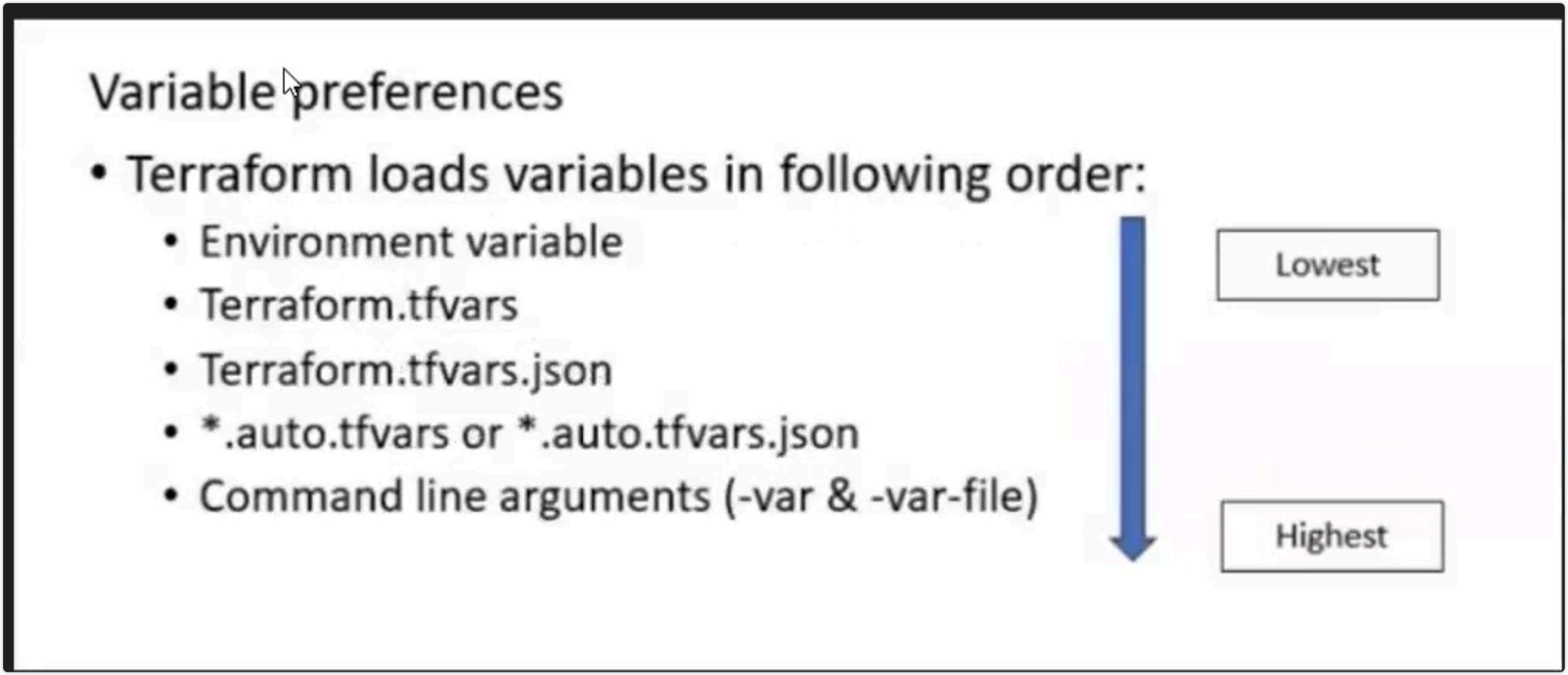
If there are many variable values to input, we can define them in a variable definition file. Terraform also automatically loads a number of variable definitions files if they are present:

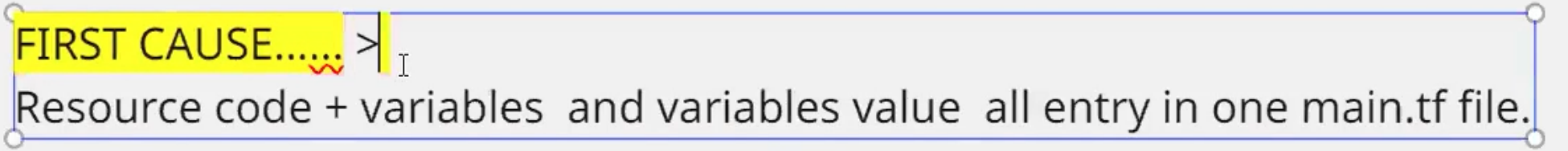


3. Environment Variables

Terraform searches the environment of its own process for environment variables named **TF_VAR_<var-name>** followed by the name of a declared variable. Terraform scans all variables starting with TF_VAR and use those as variable values for Terraform.

```
$ export TF_VAR_location=eastus
```


Variable preferences
• Terraform loads variables in following order:
• Environment variable
• Terraform.tfvars
• Terraform.tfvars.json
• *.auto.tfvars or *.auto.tfvars.json
• Command line arguments (-var & -var-file)
Lowest
Highest


FIRST CAUSE..... >
Resource code + variables and variables value all entry in one main.tf file.

```
root@cka-w2:~/project1# terraform fmt main.tf
main.tf
root@cka-w2:~/project1# cat main.tf
variable "cpu" {

  default = "t2.micro"

}



resource "aws_instance" "ram" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.nano"
  key_name      = "my_tf_key"



}
```

```
root@cka-w2:~/project1# cat main.tf
variable "cpu" {

  default = "t2.micro"

}



resource "aws_instance" "ram" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = var.cpu
  key_name      = "my_tf_key"



}
```

```
root@cka-w2:~/project1# cat main.tf
variable "cpu" {

  default = "t2.micro"

}

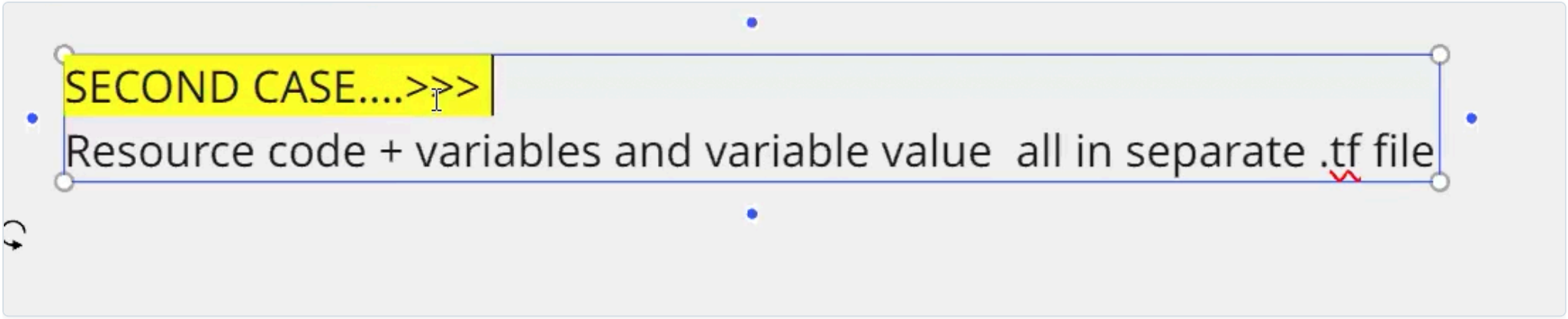


resource "aws_instance" "ram" {

  ami             = "ami-066c4849e6b3a1e3d"
 # instance_type = var.cpu
  instance_type = "${var.cpu}"
  key_name        = "my_tf_key"



}
```


SECOND CASE....>>>
Resource code + variables and variable value all in separate .tf file

```
varaible "cpu" {

        default = "t2.small"

       }
variable "ami" {

        default = "ami-066c4849e6b3a1e3d"

}

variable "key" {

        default = "my_tf_key"

}
```

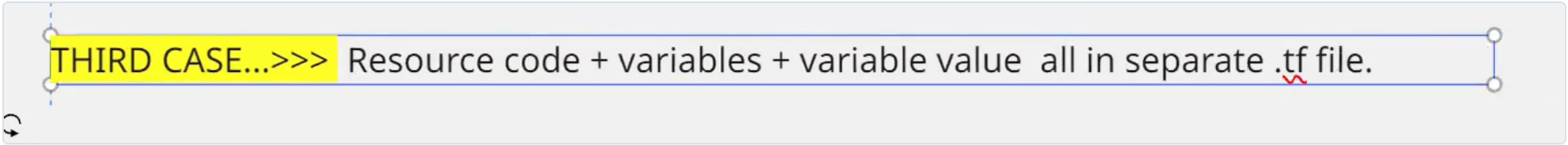
root@cka-w2: ~/project1

```
resource "aws_instance" "ram" {

        ami = var.ami
        instance_type = var.cpu
        kay_name    = var.key

}
~
```

```
root@cka-w2:~/project1# ls -l
total 8
-rw-r--r-- 1 root root  98 Sep 28 17:33 main.tf
-rw-r--r-- 1 root root 157 Sep 28 17:30 variable.tf
root@cka-w2:~/project1#
```


THIRD CASE...>>> Resource code + variables + variable value all in separate .tf file.

```
root@cka-w2:~/project1# nl main.tf
     1  resource "aws_instance" "ram" {

     2    ami             = var.ami
     3    instance_type = var.cpu
     4    key_name       = var.key

     5  }
root@cka-w2:~/project1# nl variable.tf
     1  variable "cpu" {}

     2  variable "ami" {}

     3  variable "key" {}
```

```
root@cka-w2:~/project1# nl main.tf
     1  resource "aws_instance" "ram" {

     2    ami           = var.ami
     3    instance_type = var.cpu
     4    key_name      = var.key

     5  }
root@cka-w2:~/project1# nl variable.tf
     1  variable "cpu" {}

     2  variable "ami" {}

     3  variable "key" {}




root@cka-w2:~/project1# nl terraform.tfvars
     1  ami = "ami-066c4849e6b3a1e3d"
     2  cpu = "t2.nano"
     3  key = "my_tf_key"
root@cka-w2:~/project1#
```

```
[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "ram" {
     2    ami           = var.ami
     3    instance_type = var.cpu
     4    key_name      = var.key
     5  }

[root@Devops-server project1]# nl variable.tf
     1  variable "cpu" {}

     2  variable "ami" {}
     3  variable "key" {}

[root@Devops-server project1]# nl prod-team.tfvars
     1  cpu="t2.larg"
     2  ami="ami-0cca134ec43cf708feefefwer"
     3  key="adam_keys"

[root@Devops-server project1]# nl dev-team.tfvars
     1  cpu="t2.micro"
     2  ami="ami-0cca134ec43cf708f"
     3  key="ram_aws"

[root@Devops-server project1]#
```

- Command line arguments (-var & -var-file)

```
[root@Devops-server project1]# terraform plan -var-file="dev-team.tfvars"
```

```
[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "ram" {
     2    ami           = var.ami
     3    instance_type = var.cpu
     4    key_name      = var.key
     5  }

[root@Devops-server project1]# nl variable.tf
     1  variable "cpu" {}

     2  variable "ami" {}
     3  variable "key" {}

[root@Devops-server project1]# nl terraform.tfvars
     1  cpu="t2.larg"
     2  ami="ami-0cca134ec43cf708feefefwer"
     3  key="adam_keys"

[root@Devops-server project1]#
```

```
[root@Devops-server project1]# terraform plan -var="cpu=t2.medium"
```

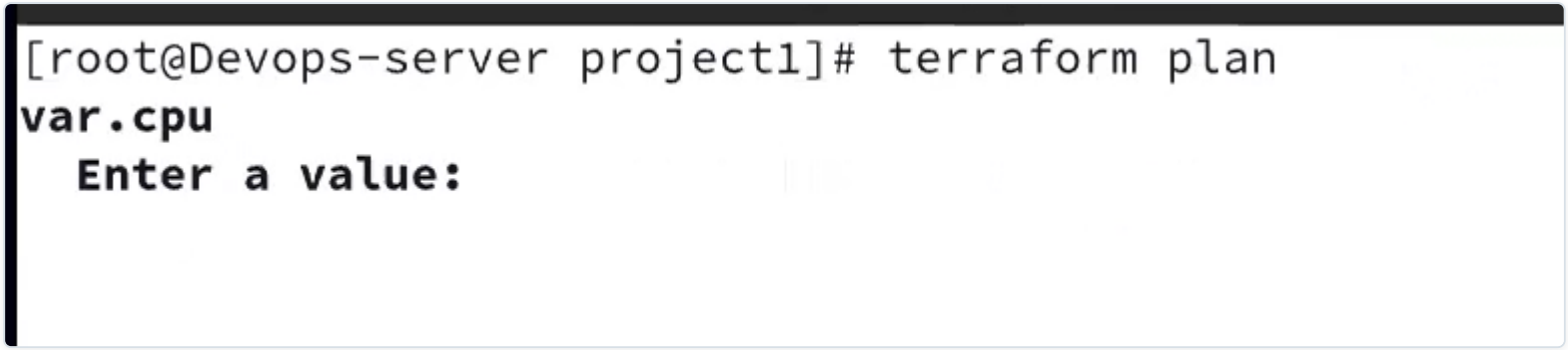
```
[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "ram" {
     2    ami             = var.ami
     3    instance_type = var.cpu
     4    key_name       = var.key
     5  }

[root@Devops-server project1]# nl variable.tf
     1  variable "cpu" {}

     2  variable "ami" {}
     3  variable "key" {}

[root@Devops-server project1]# nl terraform.tfvars
     1  ami="ami-0cca134ec43cf708feefefwer"
     2  key="adam_keys"

[root@Devops-server project1]#
```


[root@Devops-server project1]# terraform plan
var.cpu
Enter a value:

```
[root@Devops-server project1]# export TF_VAR_cpu=t2.small
```

```
[root@Devops-server project1]# nl variable.tf
     1  # define string variable example.....
     2  variable "cpu" {
     3          description = "cpu type define..."
     4          type = string
     5          default = "t2.micro"
     6  }


[root@Devops-server project1]#
```

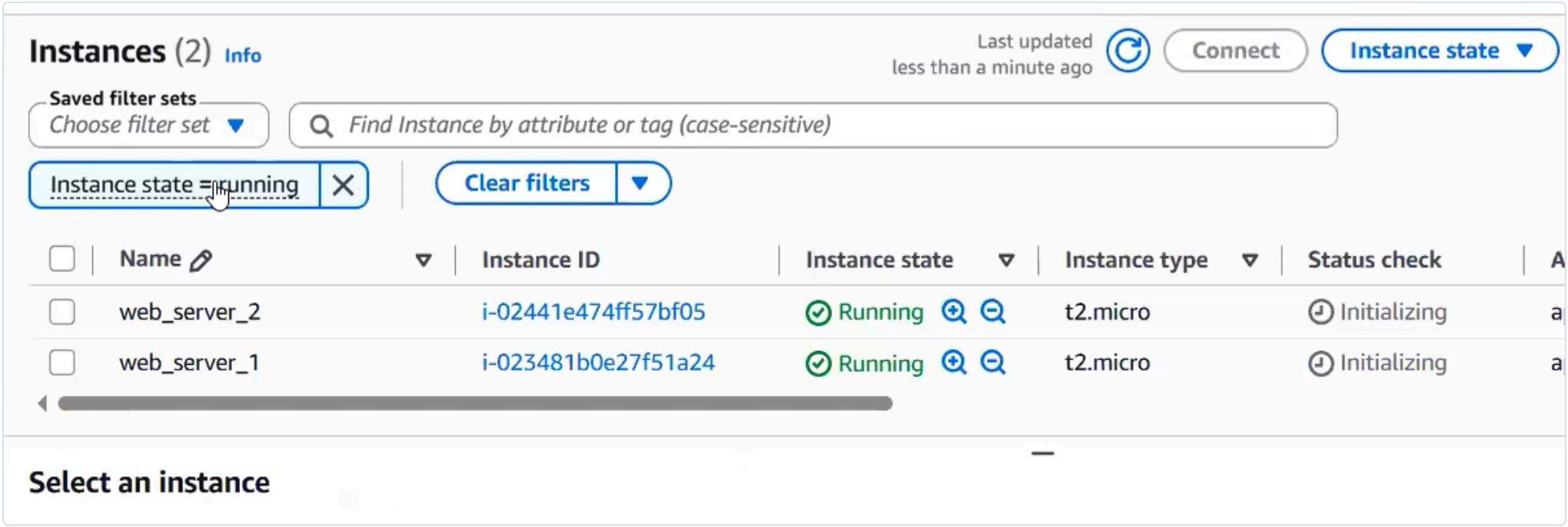
```
1 # define number variable example.....
2 variable "instance_count" {
3         description = "creat 2 instance with number string.."
4         type = number
5         default = "2"
6 }
7
~
~
```

```
[root@Devops-server project1]# nl variable.tf
     1  # define number variable example.....
     2  variable "instance_count" {
     3           description = "creat 2 instance with number string.."
     4           type = number
     5           default = "2"
     6  }

[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "ram" {
     2    ami           = "ami-0cca134ec43cf708f"
     3    instance_type = "t2.micro"
     4    key_name      = "ram_aws"
     5    count = var.instance_count
     6  tags = {
     7      Name = "web_server
     8  }
[root@Devops-server project1]#
```

Indexing pattern — `Name = "web_server_${count.index}"`




Instances (2) Info
Last updated
less than a minute ago
Connect
Instance state
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
web_server_2
i-02441e474ff57bf05
Running
t2.micro
Initializing
web_server_1
i-023481b0e27f51a24
Running
t2.micro
Initializing
Select an instance

Map variable — `amis` per region plus a `region` variable with a description


```
[root@Devops-server project1]# nl varibale.tf
     1  variable "amis" {
     2    type = map
     3    default = {
     4      "us-east-1" = "ami-b374d5a5"
     5      "us-west-2" = "ami-4b32be2b"
     6    }
     7  }

     8  variable "region" {
     9  description = "select us-east-1 or us-west-2"
    10  }

[root@Devops-server project1]#
```

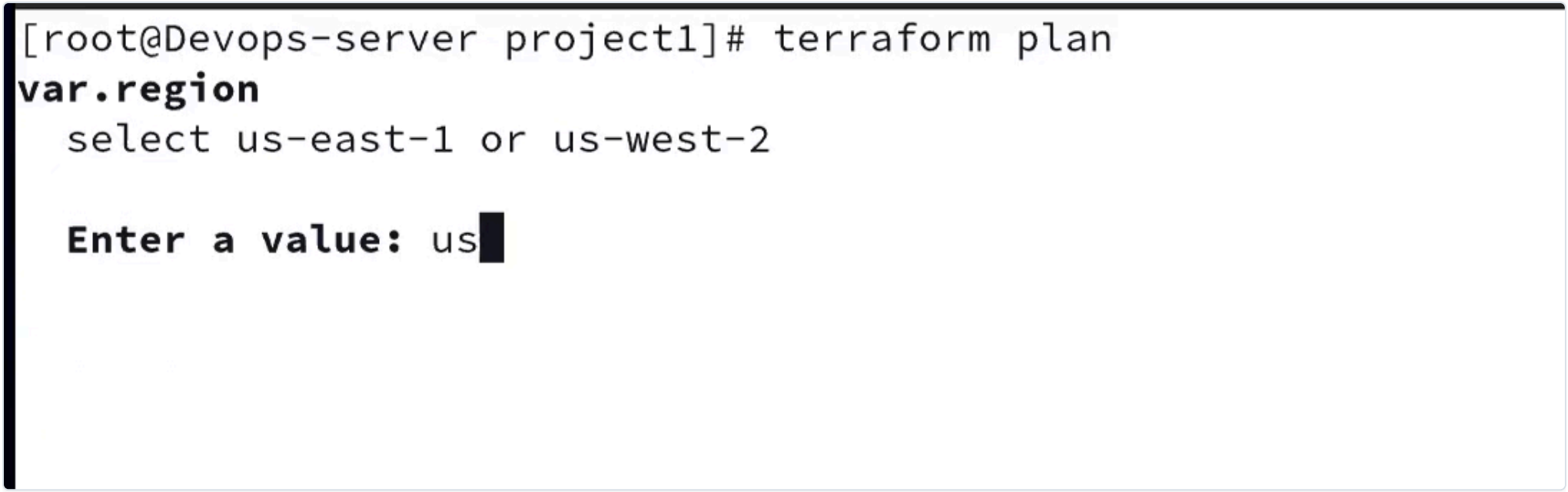
```
[root@Devops-server project1]# vim varibale.tf
[root@Devops-server project1]# vim main.tf
[root@Devops-server project1]# nl varibale.tf
     1  variable "amis" {
     2    type = map
     3    default = {
     4      "us-east-1" = "ami-b374d5a5"
     5      "us-west-2" = "ami-4b32be2b"
     6    }
     7  }

     8  variable "region" {
     9  description = "select us-east-1 or us-west-2"
    10  }

[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "example" {
     2    ami = var.amis[var.region] # This is just a dynamic way to get the right AMI ID based on the selected region.
     3    instance_type = "t2.micro"
     4  }

[root@Devops-server project1]#
```

`terraform plan` prompts for `var.region`, showing the description as help text



Output values — `instance_id` and `instance_ip` (private_ip) from `aws_instance.linux`


```
[root@Devops-server project1]# nl main.tf
     1  resource "aws_instance" "linux" {
     2    ami           = "ami-0cca134ec43cf708f"
     3    instance_type = "t2.micro"
     4    key_name      = "ram_aws"
     5    tags = {
     6      Name = "Test_server"
     7      source = "terraform"
     8    }
     9  }
    10  # check some output values..from deployment..

    11  output "instance_id" {
    12    description = "ID of the EC2 instance"
    13    value       = aws_instance.linux.id
    14  }

    15  output "instance_ip" {
    16    description = "Private IP address of the EC2 instance"
    17    value       = aws_instance.linux.private_ip
    18  }
```

```
[root@Devops-server project1]# nl main.tf
     1  locals {
     2    environment = "dev"
     3    instance_type = "t2.nano"
     4  }
     5  resource "aws_instance" "example" {
     6    ami             = "ami-0cca134ec43cf708f"
     7    key_name        = "ram_aws"
     8    instance_type = local.instance_type

     9    tags = {
    10      Name = "server-${local.environment}"
    11    }
    12  }
```